



THE POWER OF PYTHON

The Backbone of Modern Data Science



WHY PYTHON?

- Simple syntax that help researchers to focus on insights rather than complex code.
- Over 300,000 packages covering every niche from statistics to deep learning.
- Easily integrated with C, C++ and Java for heavy-tasks.
- It has many libraries that help in various places like in data cleaning, data visualization etc.





DATA SCIENCE WORKFLOW IN PYTHON

Data Manipulation

- PANDAS 
- NUMPY 

Exploratory Data Analysis (EDA)

Visulization

Matplotlib & Seaborn

Machine Learning (ML)

- Scikit-learn for classical ML
- PyTorch/TensorFlow for Deep Learning.



OOPS IN PYTHON

Help to build scalable and reusable code



CLASSES AND OBJECTS

- In Python, Classes and Objects are the fundamental components of Object-Oriented Programming (OOP)

```
class Dog:
    def __init__(self, name, age):
        self.name = name # Instance attribute
        self.age = age   # Instance attribute

dog1 = Dog("Buddy", 3)
print(dog1.name) # Output: Buddy
```

01

CLASS

In Python, Classes and Objects are the fundamental components of Object-Oriented Programming (OOP)

02

OBJECTS

In Python, Classes and Objects are the fundamental components of Object-Oriented Programming (OOP)



FOUR PILLARS OF OOP

Encapsulation

- In Python: It's like putting your data and functions inside a "capsule" (a class). You hide the messy inner workings and only show the user what they need to see. This prevents people from accidentally breaking the core logic.

Abstraction

- In Python: It's about hiding complexity. You create simple methods (like `.start()`) so the user can interact with your code without needing to understand the 500 lines of math happening behind the scenes.

Inheritance

- In Python: You can create a "Parent" class (like `Car`) and then make "Child" classes (like `ElectricCar`). The child automatically gets all the features of the parent so you don't have to write the same code twice.

Polymorphism

- In Python: It allows the same function name to do different things depending on who is using it. For example, a `.draw()` function might draw a circle for one object and a square for another.



FINAL WRAP-UP

- **Data Science:** Leverages Python's simplicity for complex analysis
 - **Encapsulation:** Keeps your data safe.
 - **Inheritance:** Saves time by reusing code.
 - **Abstraction:** Simplifies the user experience.
 - **Polymorphism:** Offers flexibility in execution.
-



THANK YOU !

From - Aryan